DOCUMENTO TÉCNICO

EdgeAI_Vision — Arquitetura e Funcionamento

1. Visão geral

O EdgeAI_Vision é uma aplicação de Visão Computacional organizada em duas partes: uma API executada no Raspberry Pi por meio de Docker e clientes Python que enviam imagens ou frames de vídeo para essa API. O modelo YOLO treinado para cinco classes de EPI é utilizado na etapa de inferência.

2. Fluxo principal

IMAGEM / VÍDEO → client_image_test.py ou client_video_test.py → HTTP → API FastAPI no Docker → main.py → inference.py → modelo YOLO/ONNX → detecções → resposta HTTP → cliente

Em termos simples: o cliente envia uma imagem; a API recebe o arquivo; o módulo de inferência executa o modelo; e o resultado retorna ao cliente.

3. Componentes e responsabilidades

Componente	Função	Papel
main.py	Implementa a API e seus endpoints HTTP.	Porta de entrada da aplicação.
inference.py	Concentra a lógica de inferência do modelo.	Processa a imagem e produz as detecções.
Dockerfile	Define como a imagem Docker da aplicação é construída.	Empacotamento do ambiente.
docker-compose.yml	Define como o serviço/container é executado.	Orquestração local.
requirements.txt	Lista as dependências Python da API.	Dependências do ambiente.
best.onnx (ONNX = Open Neural Network Exchange, formato padronizado para modelos de IA)	Modelo YOLO treinado e exportado para ONNX.	Modelo usado pelo motor de inferência.
client_video_test.py	Lê vídeo e envia frames à API.	Cliente de teste de vídeo.
client_image_test.py	Lê imagens e envia-as à API.	Cliente de teste de imagens.

4. Endpoints e comunicação

Endpoint	Uso	Retorno
/	Verificação básica da API.	Status do serviço e modo de operação.
/predict	Envia imagem para inferência.	Dados das detecções em JSON.
/predict/image	Envia imagem para inferência.	Imagem processada/anotada.

Por que existem dois endpoints? Eles oferecem duas formas de consumir a mesma inferência: /predict devolve os dados das detecções em JSON, útil para sistemas que precisam processar coordenadas, classes e confiança; /predict/image devolve a própria imagem já anotada, útil para visualização. O JSON não é automaticamente gravado em um arquivo pela API: ele é retornado na resposta HTTP e, neste projeto, o cliente pode exibi-lo ou registrá-lo se isso for programado. Os testes atuais mostram os dados no terminal quando o modo JSON é utilizado.

Os clientes não precisam conhecer a implementação interna do modelo. Eles apenas enviam arquivos via HTTP para os endpoints já existentes.

5. Implantação no Raspberry Pi

O Docker mantém a API isolada em um ambiente reproduzível. O fluxo operacional é:

1. Construir/atualizar a imagem Docker da API.
2. Iniciar o serviço com Docker Compose.
3. Verificar o serviço pelo endpoint raiz (/).
4. Executar o cliente de vídeo ou de imagens no sistema do Raspberry Pi.

Premissa adotada no projeto: a estrutura geral da API e seus endpoints permanecem inalterados; a evolução do modelo deve se encaixar nessa arquitetura.

6. Histórico resumido do treinamento YOLO

Foi utilizado o dataset SH17 para treinamento de um modelo YOLOv8n adaptado para cinco classes: pessoa, capacete, colete, luva e óculos. A preparação do dataset confirmou 6.479 imagens de treinamento e 1.620 imagens de validação.

Conjunto	Imagens	Objetos
Treinamento	6.479	33.586
Validação	1.620	8.501

O treinamento foi realizado no Google Colab com uma GPU NVIDIA Tesla T4. O modelo base utilizado foi yolov8n.pt e o treinamento inicial considerado para este projeto foi de 10 epochs.

7. Resultado após 10 epochs

Métrica	Resultado
Precision	≈ 76,7%
Recall	≈ 63,6%
mAP50	≈ 68,4%
mAP50-95	≈ 47,4%

Observação: para detecção de objetos, Precision, Recall e mAP são métricas mais apropriadas do que uma única porcentagem genérica de 'acurácia'. Em resumo: Precision indica, entre as detecções feitas pelo modelo, quantas estavam corretas; Recall indica, entre os objetos que realmente existiam, quantos o modelo conseguiu encontrar; mAP (mean Average Precision) resume a qualidade das detecções considerando precisão e localização das caixas, sendo uma das principais métricas usadas para comparar modelos de detecção. O mAP50 usa IoU de 50% como critério de correspondência, enquanto o mAP50-95 calcula a média em vários limiares de IoU, sendo portanto uma medida mais rigorosa (no mAP50, se o valor do IoU – Intersecção de área) entre a previsão do modelo e o objeto real for maior ou igual a 0,50 (50%), a detecção conta como um verdadeiro positivo..

8. Viabilidade de melhoria da precisão

O resultado obtido é um ponto de partida e há margem para evolução. O principal caminho é realizar novos treinamentos e comparar as métricas de validação, mantendo o mesmo conjunto de classes e a mesma arquitetura de integração.

- Aumentar o número de epochs, observando o comportamento de validação.
- Avaliar a qualidade e diversidade das imagens do dataset.
- Ajustar hiperparâmetros e estratégias de augmentation.
- Comparar versões do modelo e selecionar o melhor checkpoint.
- Testar o modelo final com imagens reais do cenário de uso.

  * Hiperparâmetros são configurações que controlam o treinamento, como taxa de aprendizagem, tamanho do lote (batch), tamanho das imagens e número de epochs. Augmentation é a criação de variações artificiais das imagens de treinamento, por exemplo com pequenas mudanças de escala, posição, iluminação ou espelhamento, para ajudar o modelo a generalizar melhor.

9. Onde o novo modelo se encaixa

A integração do modelo treinado não exige mudança na lógica dos clientes. O modelo exportado para ONNX (Open Neural Network Exchange, um formato padronizado para transportar modelos de IA entre diferentes ferramentas) é incorporado ao ambiente de inferência, e o inference.py continua sendo o componente responsável por utilizá-lo.

Assim, a arquitetura permanece conceitualmente:

CLIENTE → API → INFERÊNCIA → YOLO/ONNX → RESULTADO

10. Checklist de operação

- Modelo ONNX disponível no ambiente da API.
- Container Docker construído com o modelo/dependências necessários.
- docker compose up -d executado sem erros.
- Endpoint raiz respondendo como online.
- Cliente apontando para http://127.0.0.1:8000 quando executado no próprio Raspberry Pi.
- Imagem ou vídeo disponível no diretório do cliente.

11. Conclusão

O EdgeAI_Vision foi estruturado para separar claramente entrada, API, inferência e modelo. Essa separação permite substituir ou melhorar o modelo YOLO sem alterar a forma de comunicação dos clientes com a API. O modelo treinado em 10 epochs já apresenta capacidade de detecção nas cinco classes definidas, mas os resultados indicam espaço para novos ciclos de treinamento e validação antes de considerar o modelo como versão final de produção.